



UNIVERSIDAD DEL BÍO-BÍO

FACULTAD DE CIENCIAS EMPRESARIALES

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN Y
TECNOLOGÍA DE LA INFORMACIÓN

INGENIERÍA CIVIL EN INFORMÁTICA

TAREA N°2

LRC Player: Reproductor de audio con letra sincronizada

Integrantes:

Matias Constanzo Monsalve

Benjamin Valenzuela

Pablo Aguayo

Profesor:

Luis Gajardo Diaz

Fecha de Entrega:

14/06/2026

1. Introducción

El presente documento describe el desarrollo de **LRC Player**, un reproductor de audio con capacidad de mostrar letras sincronizadas en tiempo real mediante el formato `.lrc`. El proyecto implementa un analizador sintáctico (parser) generado con **SableCC** a partir de una gramática formal, demostrando la aplicación práctica de técnicas de compilación y procesamiento de lenguajes.

2. Objetivos

2.1. Objetivo General

Desarrollar un reproductor de audio que visualice letras sincronizadas a partir de archivos en formato LRC, utilizando un parser generado con SableCC.

2.2. Objetivos Específicos

- Definir una gramática formal para el formato LRC usando SableCC.
- Implementar un visitador del AST que extraiga metadatos y líneas de letra con timestamps.
- Construir una interfaz gráfica que reproduzca audio y muestre la línea de letra correspondiente según el tiempo de reproducción.
- Sincronizar la visualización de la letra con el avance del audio mediante un **Timer**.

3. Estructura del Proyecto

La siguiente figura muestra la organización de directorios y archivos del proyecto:

```
1 Tarea2/  
2   audio/  
3     Evanescence - Bring Me To Life.mp3  
4   lyrics/  
5     bring me to life.lrc  
6   lrc.sablecc
```

```
7   lrc/  
8       Reproductor.java  
9       analysis/  
10          Analysis.java  
11          AnalysisAdapter.java  
12          DepthFirstAdapter.java  
13          LyricVisitor.java  
14          ReversedDepthFirstAdapter.java  
15       lexer/  
16       node/  
17       parser/  
18   lib/
```

Listing 1: *Estructura de directorios del proyecto*

4. Gramática SableCC

La gramática definida en `lrc.sablecc` es la siguiente:

4.1. Helpers

```
1 Helpers  
2   digit = ['0' .. '9'];  
3   letter = ['a' .. 'z'] | ['A' .. 'Z'];  
4   cr = 13;  
5   lf = 10;  
6   newline = cr lf | lf | cr;  
7   apostrophe = 39;
```

Listing 2: *Helpers de la gramática*

Los helpers son macros que definen conjuntos de caracteres reutilizables:

- **digit:** cualquier dígito del 0 al 9.
- **letter:** cualquier letra mayúscula o minúscula.
- **cr** y **lf:** caracteres de retorno de carro y salto de línea.

- **newline**: secuencias que representan un salto de línea.
- **apostrophe**: el carácter comilla simple.

4.2. Tokens

```
1 Tokens
2   l_bracket = '[';
3   r_bracket = ']';
4   bcontent = (letter | digit | ' ' | '.' | ',' | '!' | '?' | '-'
              | '_' | ';' | '/' | '\\' | '(' | ')' | '#' | '$' | '%' |
              '&' | '*' | '+' | '=' | '<' | '>' | '@' | '^' | '`' | '{' |
              '|' | '}' | '~' | apostrophe | ':' )+;
5   blank = ' ' | 9 | newline;
```

Listing 3: *Tokens de la gramática*

- **l_bracket**: el carácter [de apertura.
- **r_bracket**: el carácter] de cierre.
- **bcontent**: el contenido entre corchetes. Captura timestamps (mm:ss.xx) y metadatos (ar:Artista).
- **blank**: espacios en blanco, tabulaciones y saltos de línea (ignorados).

4.3. Productions

```
1 Productions
2   lrc_file = element*;
3   element =
4       {tag_line} l_bracket bcontent r_bracket |
5       {tagged_line} l_bracket [tag]:bcontent r_bracket
        [lyric]:bcontent;
```

Listing 4: *Producciones de la gramática*

- **lrc_file**: raíz del AST, representa un archivo LRC completo.
- **tag_line**: línea con solo un tag entre corchetes (metadatos o timestamp vacío).
- **tagged_line**: línea con un tag seguido de texto de letra.

4.4. Formato LRC Soportado

```
1 [ar: Artista]
2 [ti: Título]
3 [al: Album]
4
5 [00:15.00]Primera línea de letra
6 [00:20.50]Segunda línea de letra
7 [00:25.00]Tercera línea de letra
```

Listing 5: *Ejemplo de archivo LRC válido*

4.5. Limitación de caracteres soportados

Es importante destacar que la gramática definida solo acepta caracteres pertenecientes al conjunto **ASCII estándar** (códigos 32 a 126). En consecuencia, **no se soportan caracteres especiales del español** como la letra ñ (mayúscula y minúscula), vocales con acento (á, é, í, ó, ú), diéresis (ü), ni el signo de apertura de interrogación (¿). Si el archivo `.lrc` contiene estos caracteres, el parser generado por SableCC no podrá reconocerlos correctamente y el proceso de análisis fallará. Esta limitación se debe a que el token `letter` está definido exclusivamente con las letras de la A a la Z, sin incluir caracteres extendidos.

5. Procesamiento del AST con LyricVisitor

La clase `LyricVisitor` extiende `DepthFirstAdapter` (generado por SableCC) y recorre el AST para extraer la información relevante.

5.1. Clase interna LyricEntry

```
1 public static class LyricEntry {
2     private int minutes, seconds, centiseconds;
3     private String text;
4
5     public long getMilliseconds() {
6         return (minutes * 60 * 1000L) + (seconds * 1000L) +
            (centiseconds * 10L);
7     }
8 }
```

```
7     }  
8 }
```

Listing 6: *Clase LyricEntry*

Almacena un timestamp desglosado y el texto de la línea. El método `getMilliseconds()` convierte a milisegundos para usar con `java.util.Timer`.

5.2. Métodos principales

```
1 public class LyricVisitor extends DepthFirstAdapter {  
2     private List<LyricEntry> lyrics = new ArrayList<>();  
3     private String artista = "", titulo = "", album = "";  
4  
5     private boolean isTimestamp(String s) {  
6         return s.matches("\\d+:\\d+\\.\\d+");  
7     }  
8  
9     private LyricEntry parseTimestamp(String content) {  
10        String[] parts = content.split("[:.]");  
11        int min = Integer.parseInt(parts[0]);  
12        int sec = Integer.parseInt(parts[1]);  
13        int cent = Integer.parseInt(parts[2]);  
14        return new LyricEntry(min, sec, cent, "");  
15    }  
16  
17    @Override  
18    public void caseATagLineElement(ATagLineElement node) {  
19        String content = node.getBcontent().getText().trim();  
20        if (content.contains(":")) {  
21            // Metadato: [ar:Artista], [ti:Titulo], [al:Album]  
22            int colonIdx = content.indexOf(':');  
23            String key = content.substring(0,  
24                colonIdx).trim().toLowerCase();  
25            String value = content.substring(colonIdx + 1).trim();  
26            switch (key) {  
27                case "ar": artista = value; break;  
28                case "ti": titulo = value; break;  
29                case "al": album = value; break;  
30            }  
31        }  
32    }  
33 }
```

```
29     }
30     } else if (isTimestamp(content)) {
31         // Timestamp sin letra (pausa instrumental)
32         lyrics.add(parseTimestamp(content));
33     }
34 }
35
36 @Override
37 public void caseATaggedLineElement(ATaggedLineElement node) {
38     String tag = node.getTag().getText().trim();
39     String text = node.getLyric().getText().trim();
40     if (isTimestamp(tag)) {
41         LyricEntry entry = parseTimestamp(tag);
42         lyrics.add(new LyricEntry(entry.minutes, entry.seconds,
43                                 entry.centiseconds, text));
44     }
45 }
46 }
```

Listing 7: *Métodos principales del LyricVisitor*

5.3. Flujo del visitador

```
1 Archivo LRC
2     (SableCC Parser)
3 AST (Start     ALrcFile     lista de elementos)
4     (LyricVisitor aplicado al AST)
5 Lista de LyricEntry con timestamps en ms + texto
6     (Reproductor)
7 Timer sincroniza cada linea con la reproduccion
```

Listing 8: *Flujo de procesamiento*

6. La clase Reproductor

6.1. Componentes principales

Componente	Propósito
BasicPlayer	Librería para reproducir MP3
java.util.Timer + TimerTask	Programar la aparición de cada línea de letra
JFrame + JLabel	Interfaz gráfica que muestra la letra centrada
LyricVisitor	Visitador que analiza el AST generado por SableCC

Tabla 1: *Componentes principales del reproductor*

6.2. Métodos principales

6.3. Flujo del programa

```

1 public static void main(String args[]) {
2     // 1. Determinar rutas de archivos
3     //     - 2 argumentos: MP3 y LRC (rutas explicitas)
4     //     - 1 argumento: MP3 en audio/, LRC se deriva en lyrics/
5     //     - 0 argumentos: valores por defecto
6
7     Reproductor reproductor = new Reproductor();
8     reproductor.crearVentana();           // JFrame con JLabel
9     reproductor.parsearLRC(lrcPath);     // SableCC + LyricVisitor
10    reproductor.AbrirFichero(mp3Path);   // BasicPlayer
11    reproductor.iniciarLetra(lyrics);     // Programar TimerTasks
12    reproductor.Play();                   // Iniciar reproduccion
13 }
```

Listing 9: *Método main del reproductor*

6.4. Manejo de argumentos

- **2 argumentos:** java lrc.Reproductor cancion.mp3 cancion.lrc — rutas exactas.

- **1 argumento:** `java lrc.Reproductor cancion.mp3` — busca MP3 en `audio/` y deriva LRC en `lyrics/`.
- **0 argumentos:** usa valores por defecto `audio/cancion.mp3` y `lyrics/cancion.lrc`.

7. Compilación y Ejecución

7.1. Requisitos Previos

- Java 8 o superior
- SableCC 3.7 (solo para regenerar el parser)
- Librerías necesarias en el directorio `lib/`

7.2. Compilación

```
1 # 1. Regenerar parser (solo si se modifica la gramatica)
2 java -jar sablecc-3.7/lib/sablecc.jar lrc.sablecc
3
4 # 2. Compilar todo el codigo
5 javac -cp "lib/*:." lrc/analysis/*.java lrc/lexer/*.java
   lrc/node/*.java lrc/parser/*.java lrc/Reproductor.java
```

Listing 10: *Comandos de compilación*

7.3. Ejecución

```
1
2 # Con 1 argumento (MP3 en audio/, LRC se deriva en lyrics/)
3 java -cp "lib/*:." lrc.Reproductor "cancion.mp3"
4
5 # Con 2 argumentos (rutas explicitas)
6 java -cp "lib/*:." lrc.Reproductor "audio/cancion.mp3"
   "lyrics/cancion.lrc"
7
8 # Ejemplos reales con las canciones de prueba
```

```

9 java -cp "lib/*:." lrc.Reproductor "audio/Michael Jackson - They
    dont care about us.mp3" "lyrics/Michael Jackson - They dont
    care about us.lrc"
10
11 java -cp "lib/*:." lrc.Reproductor "audio/Michael Jackson -
    Thriller.mp3" "lyrics/Michael Jackson - Thriller.lrc"
12
13 java -cp "lib/*:." lrc.Reproductor "audio/The Police - Every
    breath you take.mp3" "lyrics/The Police - Every breath you
    take.lrc"
14
15 # Nota
16 La separacion del -cp "lib/*:." para linux y macOs funciona con
    ':', para Windows usar ';'.
17 java -cp "lib/*;." lrc.Reproductor "audio/Michael Jackson - They
    dont care about us.mp3" "lyrics/Michael Jackson - They dont
    care about us.lrc"

```

Listing 11: *Formas de ejecución*

8. Pruebas realizadas

Se probó el reproductor con las siguientes canciones:

Canción	MP3
Michael Jackson — They Don't Care About Us	audio/Michael Jackson - They dont care about
Michael Jackson — Thriller	audio/Michael Jackson - Thriller.mp3
The Police — Every Breath You Take	audio/The Police - Every breath you take.mp3

Tabla 2: *Pruebas realizadas*

```
Artista: Evanescence
Titulo: Bring Me To Life - Evanescence
Album:
Líneas de letra: 78
Jun 13, 2026 10:37:47 PM javazoom.jlgui.basicplayer.BasicPlayer open
INFO: open(Evanescence - Bring Me To Life.mp3)
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer createLine
INFO: Create Line
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer createLine
INFO: Create Line : Source format : MPEG1L3 44100.0 Hz, unknown bits per sample, stereo, unknown frame size, 38.28125 frames/second
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer createLine
INFO: Create Line : Target format: PCM_SIGNED 44100.0 Hz, 16 bit, stereo, 4 bytes/frame, little-endian
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer createLine
INFO: Line : com.sun.media.sound.DirectAudioDevice$DirectSDlg757942a1
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer startPlayback
INFO: startPlayback called
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer initLine
INFO: initLine()
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer openLine
INFO: Open Line : BufferSize=88200
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer openLine
INFO: Master Gain Control : [-80.0,6.0206] 0.625
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer openLine
INFO: Pan Control : [-1.0,1.0] 0.0078125
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer startPlayback
INFO: Creating new thread
Jun 13, 2026 10:37:48 PM javazoom.jlgui.basicplayer.BasicPlayer run
INFO: Thread Running
Bring Me To Life
(feat. Paul McCoy)
lrc by @negaonie_
How can you see into my eyes
like open doons?
Leading you down into my core
where I've become so numb
Without a soul
my spirit's sleeping
```

Figura 1: *Esquema de la interfaz del reproductor en la terminal*

```
How can you see into my eyes
```

Figura 2: *Esquema de la interfaz gráfica del reproductor*

9. Conclusión

Se ha implementado exitosamente un reproductor de audio con sincronización de letras en formato LRC. Los principales logros del proyecto son:

- Diseño e implementación de una gramática formal para LRC usando SableCC con solo 24 líneas de especificación.
- Generación automática de un parser funcional a partir de la gramática.
- Implementación de un visitador para extraer información estructurada del AST.

- Sincronización precisa entre la reproducción de audio y la visualización de letras.
- Separación clara entre la definición de la gramática, la lógica de extracción y la integración con audio/UI, facilitando el mantenimiento.

El proyecto demuestra la utilidad de las herramientas de generación de parsers como SableCC para procesar lenguajes específicos de dominio (DSL), así como la integración de componentes de análisis sintáctico, reproducción multimedia e interfaz gráfica de usuario.

Anexos

Anexo A: Gramática completa

La gramática completa se encuentra disponible en el archivo `lrc.sablecc` del proyecto.

Anexo B: Código fuente completo

El código fuente completo está disponible en el repositorio del proyecto, incluyendo todas las clases generadas por SableCC y las implementaciones personalizadas.